

In the following exercises we are going to program our own implementation of a K-Means algorithms. There is a dataset1.csv

Download dataset1.csv available. This dataset includes meteorological data (weather data) from the weather station at De Bilt for the year 2000. The dataset is part of the freely available weather data gathered by the KNMI since 1901, for several weather stations in The Netherlands1.

The data that we are using has 11 attributes:

```
YYYYMMDD: date in year, months, days;

FG: day average windspeed (in 0.1 m/s);

TG: day average temperature (in 0.1 degrees Celsius);

TN: minimum temperature of day (in 0.1 degrees Celsius);

TX: maximum temperature of day (in 0.1 degrees Celsius);

SQ: amount of sunshine that day (in 0.1 hours); -1 for less than 0.05 hours;

DR: total time of precipitation (in 0.1 hours);

RH: total sum of precipitation that day (in 0.1 mm); -1 for less than 0.05mm.
```

Data is entered in a comma-separated format, which can be easily loaded into Python by using the numpy-module (the use of numpy is recommended as it supplies you with valuable additions to work with vectors and arrays). Use the following code-excerpt to import the data into Python:

```
import numpy as np

data = np.genfromtxt('dataset1.csv', delimiter=';', usecols=[1,2,3,4,5,6,7], converters={5: lambda s: 0 if s == b"-1" else float(s), 7: lambda s: 0 if s == b"-1" else float(s)})
```

Note that the converters are needed to transform the -1's in the columns SQ (hours sunshine) and RH (amount of rain) to a more useful 0. This is a minor correction to make the data better represent what it means, improving the distance calculations between different data points.

We deliberately skip column 0 since that contains our date, which we will use to create the labels for the data. Instead of using the year, month, day as label, we will generalise to seasons, which we do as follows2:

```
dates = np.genfromtxt('dataset1.csv', delimiter=';', usecols=[0]) labels = []
for label in dates:
    if label < 20000301: labels.append('winter')
    elif 20000301 <= label < 20000601: labels.append('lente')
    elif 20000601 <= label < 20000901: labels.append('zomer')
    elif 20000901 <= label < 20001201: labels.append('herfst')
    else: # from 01-12 to end of year
        labels.append('winter')
Exercise
```

We are going to re-cluster the available data. This would show us whether we can indeed find 4 different groups in the data (one for each season). We again need to import the data and labels as we did above (we

will need the labels later to verify whether the clusters made by K-Means indeed reflect the true season of the data points).

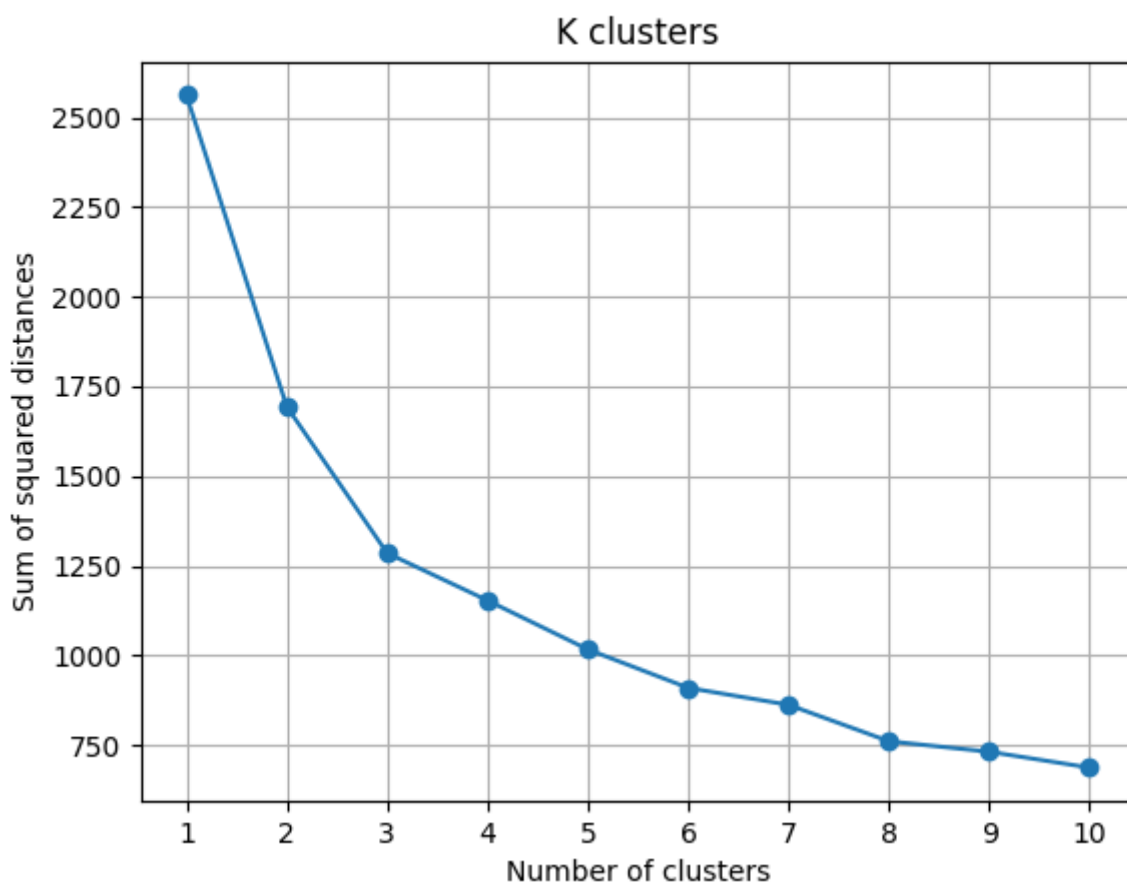
Write a bare metal k-Means clustering implementation, to cluster the data in k groups.

How many clusters can you (reliably) detect? Use a scree plot to determine the optimal size of k for this dataset. Include the determined value of k in the documentation included with the code.

Hints:

Don't forget to apply normalisation.
(it is not mentioned in the reader, but it's essential nevertheless - ask your teacher if you missed that)
Start the x axis of the scree plot with 1.
In the scree plot, select the point where the slope changest fastest.
Don't forget to work with squared distances.

Answer



The 'Elbow Point' seems to be at 3 clusters, not necessarily four. This is probably caused by the fact that a Dutch autumn and a Dutch spring really aren't that different. It might be more effective to group this into a "transitional" season. Well, if you let computer science majors write the seasons, anyway.

After all, in a broad view, what is the difference in absolute elevation change between going uphill, and going downhill?